# Asterism: A Hebbian-Weighted Knowledge Graph as a Priority Index over Long-Term LLM Memory

**Bidit Das**
Independent Researcher
github.com/biditdas18/asterism

## Abstract

Large language model (LLM) assistants increasingly persist user information across sessions as unstructured "memory" summaries. Such summaries behave like an unindexed heap: every stored fact carries equal retrieval priority, and nothing signals which facts currently matter most to the user. We present Asterism, a local-first system that builds a weighted, decaying knowledge graph from conversation history and injects a ranked subset of it into the model's context at inference time. The central design claim is architectural: the graph functions as a priority index over flat memory, in the same sense that a B-tree indexes an ordered file, with node weight acting as a salience signal analogous to an index key. We evaluate the mechanism on a seeded personal graph across five retrieval conditions and three assistant models (two Claude models and one GPT model), with decoding held fixed and the graph reseeded before every call, and report a specific, bounded finding: structured memory's advantage over a flat list is not recall but *decisiveness under ambiguity*, and it is a property of the structure itself rather than of any instruction to prioritize. On priority-ranking queries scored with a pre-registered, held-out-validated commit/hedge metric, the two Claude models commit to a weight-justified answer more often under the weighted graph than under a flat list of the same facts, in every one of five runs each (sign-test $p = 0.03$); a control condition that adds the graph's prioritization *instruction* to the flat list closes none of that gap, isolating the effect to structure. We further show the decisiveness is *accurate*: when the graph condition commits, it commits to the actually-highest-priority item substantially more often than the flat or no-memory conditions do, so the added commitment is not confident noise. The effect is model-dependent: it is robust in both Claude models and absent in GPT, which commits at a constant elevated rate regardless of structure. We characterize the instrument itself, reporting scorer–human agreement ($\kappa = 0.64$) whose errors are directional and bias the measured gap toward the null. We also characterize two engineering components required to make the graph trustworthy as an index: an entity-resolution layer whose accuracy ceiling we measure empirically, and a supersession mechanism that demotes contradicted facts from default retrieval. Local retrieval latency scales approximately linearly, remaining near 34 ms at 10,000 nodes. We release the full system, benchmarks, and negative results.

## 1 Introduction

Conversational LLM systems now routinely retain information about a user between sessions. The dominant form this retention takes is a running natural-language summary: a list of facts, preferences, and history that is prepended to the model's context on later turns. This format is simple and effective for recall, but it is structurally flat. Every fact sits at the same level. There is no representation of which facts are central to the user and which are incidental, no representation of how facts relate, and no representation of which facts have been superseded by newer ones.

In database terms, a flat memory summary is an *unindexed heap*: a store you can scan but cannot query by priority. The contribution of this paper is to take that analogy literally and build the missing index. Asterism constructs a weighted directed graph from a user's conversation history, where nodes are concepts and entities, edges are relations, and edge and node weights encode how frequently and recently the user has engaged with each. At inference time, the highest-weight region of the graph is serialized into the model's context. The weight ordering is the index: it tells the retrieval step which facts to surface first, exactly as an index key tells a query planner where to look before touching the underlying data.

This framing makes a falsifiable prediction. If

the graph is genuinely acting as a priority index and not merely as a memory store, then its benefit over an unstructured list of the same facts should appear specifically on tasks that require prioritization, and should largely vanish on tasks that require only recall. We test this directly, and we design the test to survive the obvious confounds. The graph condition, as deployed, both imposes structure and instructs the model to prioritize; a naive comparison cannot tell these apart. We therefore run five conditions against the same underlying facts: the weighted graph, an unweighted flat list, no memory, a flat list that adds the graph's prioritization *instruction* without its structure, and a graph presented *without* that instruction. The flat-versus-graph gap isolates structure from having memory at all; the two added conditions isolate structure from instruction. We hold decoding fixed across models, reseed the graph before every call so no response conditions on state a prior response wrote, and repeat the whole design across three assistant models and five runs each.

We find the gap is real but narrow, and report its shape precisely. The graph does not help the model recall more; a flat list recalls the same facts. The graph helps the model *decide*: on queries asking what to prioritize, the flat list sees every fact but has no salience signal, so it hedges and returns the decision to the user, while the weighted condition commits and justifies by weight. Three results sharpen this. The effect is structural, not instructional: adding the prioritization instruction to the flat list does not reproduce it, while the instruction-free graph does. It is *accurate*: the graph condition commits to the genuinely highest-priority item far more often than the alternatives, so it is not merely confident. And it is model-dependent: robust across both Claude models, absent in the GPT model, which is already decisive without structure. This scoping identifies exactly where, and for which models, graph structure pays for itself.

Beyond the core comparison, an index over a live, automatically-populated store raises two engineering problems that we treat as first-class. First, the entities extracted from conversation are not canonical: the same concept surfaces under many surface forms, and without resolution the graph fragments into near-duplicate nodes, corrupting the weight signal. We implement resolution and, more usefully, *measure its ceiling*: we show that off-the-shelf sentence embeddings cleanly merge reworded variants but cannot reliably separate true synonym substitutions from merely related concepts at any single similarity threshold. Second, facts change. A user who switches tools or reverses a decision leaves a stale edge in the graph that, absent intervention, competes with the new fact for retrieval. We add a supersession mechanism that demotes contradicted edges from default injection while retaining them for explicit history queries.

Our contributions are:

1. A framing of a weighted personal knowledge graph as a priority index over flat LLM memory, with node weight as the salience key (Section 3).

2. A five-condition, three-model evaluation that isolates the index's value from the value of memory per se *and* from the prioritization instruction, yielding a specific finding: graph structure aids decisiveness under ambiguity, not recall; the decisiveness is accurate rather than merely confident; and the effect is robust across Claude models but absent in GPT (Section 4).

3. An empirical characterization of the entity-resolution ceiling for embedding-based canonicalization of short concept labels, including the negative result (Section 5).

4. A supersession mechanism for demoting contradicted facts from retrieval, and a latency study showing approximately linear scaling to 10,000 nodes (Sections 6, 7).

All code, benchmarks, and the seeded evaluation graph are publicly available.

## 2 System Overview

Asterism is a local-first Python system. The entire graph lives in a single SQLite file on the user's machine; no data leaves the device except the user's own prompt and the injected context they choose to send to the model. Figure 1 shows the pipeline.

**Extraction.** Each conversational exchange is processed by a small extraction model that emits (source, relation, target) triples. Sources and targets become nodes; relations become labeled, weighted edges.

**Weighting and decay.** Every node and edge carries a scalar weight. Repeated engagement increases weight; disuse decreases it. Following the

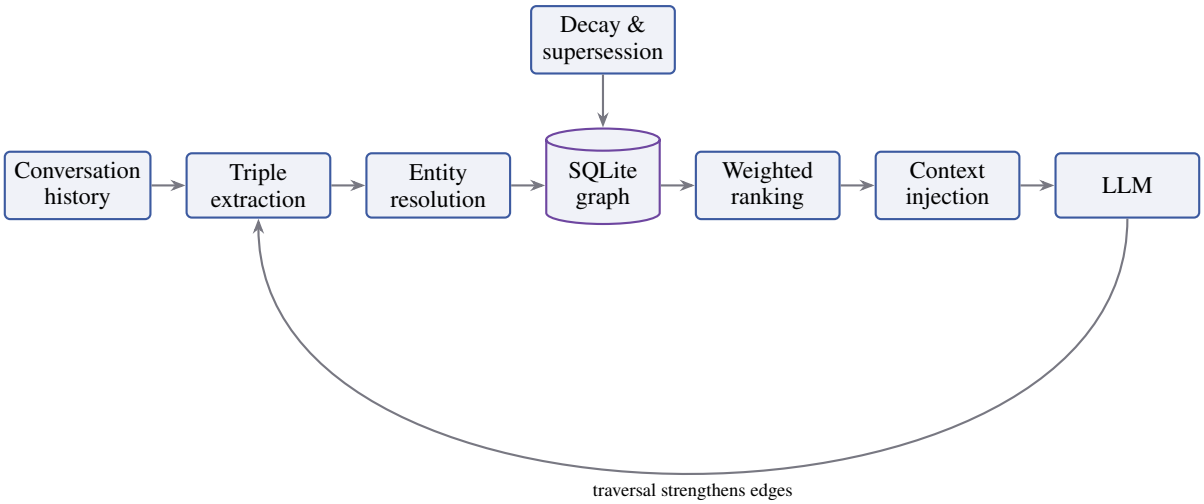


Figure 1: Asterism pipeline. Conversation history is extracted into (source, relation, target) triples, canonicalized by entity resolution, and written to a local SQLite graph. A background process applies decay and supersession. At inference time the highest-weight subgraph is ranked and injected into the model's context. When the model traverses a relation in its response, the corresponding edge is strengthened, closing a Hebbian feedback loop.

Hebbian principle that co-activated connections strengthen [1], an edge is reinforced whenever the model traverses it while answering a query. The complementary process is decay: an edge or node that accumulates session exposure without being traversed loses weight and is eventually pruned. Decay is measured in active session time rather than wall-clock time, so a concept does not fade while the user is simply away. This weight signal, accumulated over many sessions, is what the index ranks on.

**Retrieval.** On each turn, the graph is queried for its highest-weight nodes and their relations, which are serialized into a compact block and placed in the model's system context. This is the read path of the index. We deliberately keep it simple: an ordered scan by weight, returning the top-$k$ nodes. Section 7 shows this is inexpensive even at ten thousand nodes.

## 3 The Graph as a Priority Index

The organizing idea of this work is that the structures a database uses to make an unindexed file queryable map directly onto the problem of making flat LLM memory prioritizable. We make the correspondence explicit because it drives the design and the evaluation.

A flat memory summary is a heap file: an append-only list of facts with no access structure. You can read all of it, but you cannot ask it "what are the ten most important facts about this user" without scanning everything and imposing an ordering after the fact. A B-tree [2] solves the analogous problem for ordered files by maintaining a separate structure whose keys let a reader locate relevant records in logarithmic rather than linear cost. The key is not the data; it is a priority-ordered pointer into the data. In everyday terms, the difference is the index at the back of a book versus reading every page to find a topic: the index does not contain the book's content, but it tells you which pages matter and in what order to consult them.

Asterism's weighted graph plays the role of that index over the heap of conversational facts, with the following correspondences:

- **Node weight ↔ index key.** Weight is the quantity the retrieval step orders on. A high-weight node is a high-priority pointer: it is surfaced first, just as a B-tree directs a reader to the most relevant page before touching the file.
- **Graph traversal ↔ query planning.** Answering a query by walking weighted relations is a plan over the memory space, in contrast to a full scan of the flat summary.
- **Decay ↔ eviction policy.** Pruning low-weight, unused nodes is a cache-eviction discipline: it keeps the working index small and

current, so retrieval priority reflects present rather than historical relevance.

- **Underlying facts persist.** Crucially, decaying a node removes it from the *index*, not from the record of what was said. The distinction mirrors dropping an index entry without deleting the row.

We stress that graph-structured memory for LLMs is not itself new; systems such as Mem0 include graph-based memory representations [3], and MemGPT frames context management as an operating-system memory hierarchy [4]. Our claim is narrower and specific: that the *weighting* of such a graph should be understood and evaluated as an index key, and that doing so predicts precisely where the structure helps. The next section tests that prediction.

## 4 Core Evaluation: Index Structure versus Flat Recall

### 4.1 Design

Evaluating a memory system risks conflating three different benefits: the benefit of having memory at all, the benefit of the particular structure imposed on it, and the benefit of instructing the model to act on that structure. The graph condition as deployed carries all three: it supplies facts, it imposes a weighted structure, and its system prompt instructs the model to prioritize high-weight nodes. To separate them, we run five conditions against the *same underlying facts*:

1. **Graph.** The top-weighted subgraph is injected, preserving weight ordering and relational structure, with an instruction to prioritize high-weight nodes (the deployed condition).

2. **Flat list.** Every node label is injected as an unordered, unweighted list. The same facts are present; only the structure and salience signal are removed.

3. **Flat list + instruction.** The same unordered list, but its prompt adds the same prioritize-and-commit instruction the graph carries, with no weights and no structure. This isolates the *instruction*.

4. **Graph, neutral.** The same weighted structure as GRAPH, but with the explicit prioritize/commit instruction removed and structure presented plainly. This isolates the *structure*.

5. **None.** No memory is injected.

Two contrasts do the work. GRAPH vs. FLAT LIST isolates structure-and-weighting from information content, since both see identical facts. GRAPH, NEUTRAL vs. FLAT LIST, together with FLAT LIST + INSTRUCTION vs. FLAT LIST, separates the effect of the structure from the effect of the instruction: if the effect is instructional, the instruction added to a flat list should reproduce it and the instruction-free graph should lose it; if it is structural, the reverse. NONE is a reference point.

We evaluate on a seeded personal graph of 50 nodes organized into weighted domain/theme/concept chains, representative of a moderately active user's accumulated history. We focus the quantitative comparison on *priority queries*: twelve questions that ask what the user should focus on, what matters most, or which of two things to prioritize, each requiring the model to choose or rank rather than merely recall.

To keep the comparison clean across a large call budget we fix three things. Decoding temperature is pinned to a single value on every backend, so cross-model differences cannot be attributed to sampling settings. The graph is reseeded from the pristine seed before *every* individual call, so no response conditions on graph state that an earlier call's write-back mutated; an invariant check aborts a call rather than run it if the pre-call seed ever drifts. And we repeat the entire five-condition design across three assistant models — two Claude models (Sonnet and Opus class) and one GPT model — with five independent runs each, reporting the mean and range over runs rather than a single draw. The graph-maintenance extractor is a separate, fixed model and is held constant across all conditions, so the assistant model is the only moving part.

### 4.2 Metric

We score each response with a single binary metric: does it COMMIT to a specific ranked answer, or HEDGE? A response commits if it names a single top priority or makes an explicit choice and justifies it; it hedges if it defers the decision to the user, enumerates options without selecting, or answers "it depends." The classifier is a deterministic rule, preregistered and frozen before the evaluation run, and validated on a held-out set of eleven hand-labeled responses drawn from earlier development runs

(not from the evaluation set), on which it achieved 11/11 agreement. We report the frozen classifier's output without post-hoc adjustment. Restricting the metric to priority queries is deliberate: commit-versus-hedge is only meaningful for questions that call for a ranking. Descriptive recall queries, for which a flat list is sufficient by construction, do not admit this metric and are excluded from it.

Because the entire dependent variable rests on this classifier, we validate it against human judgment on the actual evaluation data, not only the development set. One author independently labeled all 36 responses of one full run (twelve queries across the three core conditions) as commit or hedge without seeing the classifier's labels, and we computed agreement. Raw agreement was 83% and Cohen's $\kappa$ was 0.64 (substantial). The disagreements are directional and, importantly, favor conservatism: of six disagreements, the classifier labeled as HEDGE four graph-condition responses a human read as commitments (for instance, a bare imperative "Ship the package" that the frozen rule's vocabulary did not cover), and over-credited two flat-list responses. Both directions understate the graph condition relative to the flat list, so the measured graph−flat gap is a conservative estimate of the true difference. We did not retune the classifier after seeing these disagreements; doing so would fit it to the evaluation data. The scorer–human comparison sheet is released with the code.

### 4.3 Findings

Table 1 reports the result. Read the two Claude models first. For both, the graph conditions commit more than the flat list: GRAPH exceeds FLAT LIST in all five runs for each model (a sign test on 5/5 gives $p = 0.03$), and NONE commits least, confirming only that memory helps, which was never in question.

**The effect is structural, not instructional.** The two control conditions settle what the raw gap cannot. Adding the prioritization instruction to a flat list (FLAT + INSTR.) does *not* reproduce the effect: for Sonnet it is identical to the plain flat list (3.6 vs. 3.6), and for Opus it lands well below the graph. Conversely, removing the instruction from the graph (GRAPH, NEUTRAL) does not destroy the effect and for Sonnet strengthens it (8.8 vs. 5.4 for the instructed graph). So it is the weighted structure, not the instruction to use it, that produces decisiveness; telling a model to prioritize a flat list does little, while giving it a weighted structure and no instruction does a lot. (Sonnet's GRAPH, NEUTRAL exceeding its instructed GRAPH suggests the explicit instruction can even interfere once structure is present; this appears only for Sonnet among the three models and we treat it as a single-model observation, not a general claim.)

**The decisiveness is accurate, not merely confident.** A metric that rewards committing could reward confident error: committing to the *wrong* priority is worse than an honest hedge. Because the seed graph has known weights, the correct top-priority item is defined, so we can check what each committed answer commits *to*. Among committed responses, the graph conditions commit to the actually-highest-priority item far more often than the flat or no-memory conditions: 46% vs. 15% (Sonnet), 66% vs. 26% (Opus), 71% vs. 53% (GPT). The added decisiveness is thus targeted at the right answer, not noise. The effect is nonetheless *relative*: even the graph conditions are wrong a substantial fraction of the time (Sonnet's graph is correct on 46% of its commits), so structure improves *which* answer the model commits to without making it an oracle. The finding is the gap between conditions, not the absolute ceiling.

**The effect is model-dependent.** The Claude pattern does not appear for GPT: its four memory-bearing conditions are statistically indistinguishable (6.6–7.0), only NONE is clearly lower, and the graph−flat gap is negative in two of five runs ($p = 0.69$, vs. 5/5 and $p = 0.03$ for the Claude models). The mechanism is visible in NONE: GPT commits under no-memory more than either Claude model (2.2 vs. 0.0 and 1.0), typically to a generic advice template rather than a user-specific answer. Being decisive at baseline, it has little room for a salience signal to raise its commit rate — though, per the correctness result, structure still improves *what* it commits to. We report this as a boundary condition: structured memory supplies decisiveness to models that hedge by default, which the Claude models do and the GPT model does not.

Qualitatively, the flat-list and no-memory hedges differ in a way worth noting. No-memory responses hedge honestly ("I don't have your priorities"); flat-list responses, which hold the same facts the graph does, instead hedge through filler and by reflecting the ranking question back to the user ("great question, but which of these feels most urgent?"). The graph's contribution is the salience

| Model | None | Flat list | Flat + instr. | Graph | Graph, neutral |
|---|---|---|---|---|---|
| Claude Sonnet | 0.0 [0–0] | 3.6 [3–5] | 3.6 [3–4] | 5.4 [3–8] | **8.8 [7–11]** |
| Claude Opus | 1.0 [0–2] | 1.6 [0–3] | 3.6 [2–5] | **6.2 [4–9]** | 5.8 [4–8] |
| GPT | 2.2 [1–4] | 7.0 [6–8] | 6.6 [3–10] | 6.6 [3–9] | 6.6 [5–9] |

Table 1: Commit rate on twelve priority queries, mean [min–max] over five runs, by model and condition. Decoding is pinned identically across models and the graph is reseeded before every call. All conditions are scored by the same frozen, held-out-validated classifier. FLAT + INSTR. adds the graph's prioritization instruction to the flat list (instruction without structure); GRAPH, NEUTRAL removes that instruction from the graph (structure without instruction).

signal that lets the model resolve the question rather than perform helpfulness around it.

The finding, stated precisely, is that a weighted memory structure's contribution is not recall but *accurate decisiveness under ambiguity*, that this is a property of the structure rather than of an instruction to use it, and that it holds robustly for the Claude-family models we test while being absent for GPT. This is a narrower claim than "graph beats flat memory," and we prefer it because it is diagnostic: it tells a system designer that the index earns its complexity precisely when the application must prioritize, rank, or choose, for models that do not already commit by default.

## 5 Entity Resolution and Its Ceiling

An index is only as good as the keys it orders. Because Asterism populates its graph automatically from extracted triples, the same real-world concept arrives under many surface forms across sessions. Without canonicalization, "AI memory tools," "AI memory tooling," and "AI Memory Tools" become three separate nodes, each accumulating a fraction of the weight the single underlying concept deserves. Fragmentation directly corrupts the index: it splits the salience signal and can prematurely decay a concept that is, in aggregate, highly active.

### 5.1 Two-stage resolution

We resolve labels at write time in two stages. The first is a normalized exact/prefix match: labels are lowercased and whitespace-collapsed, and a new label that exactly matches or is a normalized prefix of an existing one is merged into it. This is cheap and precise, and against our live label set it caught the clear duplicate variants without error.

The prefix rule has an obvious ceiling: it cannot merge variants that share meaning without sharing a prefix. To reach those, the second stage falls through to embedding similarity, computing cosine similarity between label embeddings from a compact local model [5] and merging above a calibrated threshold. The model runs offline on CPU, consistent with the system's local-first design, and is invoked only on the write path, so it does not affect retrieval latency.

| Label pair | Sim. | Merge? |
|---|---|---|
| *Reworded, shared vocabulary* | | |
| CLI Tools / CLI tool | 0.982 | yes |
| AI Memory Tools / AI memory tooling | 0.969 | yes |
| LLM context limitations / . . . window limitations | 0.954 | yes |
| *True synonym substitution* | | |
| LLM requires GPUs / Large Language Models need graphics processors | 0.745 | no |
| *Distinct but related (must not merge)* | | |
| Philosophy & Identity / Research Identity | 0.77–0.88 | no |

Table 2: Embedding similarity on short concept labels. Reworded variants that reuse vocabulary score high and merge correctly. True synonym substitutions (different words, same meaning) score in the same range as genuinely distinct concepts, so no single threshold separates them.

### 5.2 The measured ceiling

The useful result here is not that embedding resolution works but where it stops working, which we measured directly against the live graph.

Table 2 shows the pattern, which held across every embedding model we tested. Reworded or reordered labels that reuse the same vocabulary score high (0.92–0.98) and merge correctly. But *true synonym substitution* — different words for the same thing — scores low (roughly 0.58–0.80) and lands in the same range as pairs that are merely topically related and must not be merged. The canonical hard case, "LLM requires GPUs" versus "Large Language Models need graphics processors," scores only 0.745. A threshold aggressive enough

to catch it would also collapse distinct concepts.

We calibrated the operating threshold to 0.92, which the data show is the point below which false merges begin. This choice makes the resolver conservative: it eliminates the fragmentation caused by rewording, which is the common case in practice, while explicitly declining to merge synonym pairs it cannot distinguish from related-but-distinct ones. We regard the honest ceiling as more useful than an aggressive threshold that would silently corrupt the graph. Closing it properly requires supervision — either a fine-tuned equivalence model or an LLM-based semantic check — which we leave to future work.

## 6 Supersession: Demoting Contradicted Facts

An index over a live store must handle facts that change. When a user who "uses Postgres" switches to SQLite, naive decay eventually fades the old edge, but until it does, both facts coexist in the graph and both can be retrieved as current. The result is a stale entry competing with a fresh one for a place in the injected context.

We add an explicit supersession mechanism scoped to the narrow, detectable case: a new edge from the same source, with the same relation, to a different target. On such a conflict the prior edge is marked as superseded by the new one via a nullable self-reference on the edge, and its time-to-live is dropped so that it becomes eligible for pruning rather than lingering. Superseded edges are excluded from default context injection, so the model sees only the current fact; a node is withheld from injection only when *every* fact-edge pointing to it has been superseded, so a concept with any live relation still appears. The superseded edges are not deleted. They remain in the store and are recoverable for explicit history queries of the form "what did I used to use," preserving the distinction the flat summary cannot represent: the difference between a fact that is outdated and one that was never true.

This narrow rule — same source, same relation, different target — is deliberately conservative. It does not attempt open-ended contradiction detection across unrelated relations, which would risk demoting facts that only appear to conflict. The scoped version is sufficient to demonstrate the mechanism: adding a conflicting edge correctly demotes its predecessor from retrieval while leaving unrelated relations on the same source untouched.

| Nodes | Mean (ms) | P95 (ms) | Max (ms) |
|---|---|---|---|
| 102 | 0.34 | 0.49 | 0.52 |
| 1,006 | 2.81 | 2.62 | 14.42 |
| 10,051 | 33.87 | 38.70 | 42.84 |

Table 3: Local retrieval latency by graph size, retrieval step only (no model API call), 20 runs per scale. A roughly $99\times$ increase in nodes yields a roughly $100\times$ increase in mean latency: approximately linear scaling.

## 7 Latency and Scaling

For the graph to serve as an index without harming interactive use, the read path must stay cheap as the graph grows. We benchmarked the retrieval step — loading and weight-ordering the nodes and edges for injection — in isolation, excluding any model API call, at three graph sizes. Synthetic graphs were shaped like real usage (weighted domain/theme/concept chains rather than flat random nodes), and each scale was measured over 20 runs.

Table 3 shows the result. Increasing the graph by a factor of roughly 99 (from 102 to 10,051 nodes) increases mean retrieval latency by a factor of roughly 100, i.e. retrieval scales approximately linearly with graph size and does not degrade faster than the graph itself grows. In absolute terms, retrieval averages under 34 ms even at ten thousand nodes, which is negligible beside the network and inference latency of the model call it precedes. The retrieval step is therefore not a bottleneck for interactive use at the scale a personal graph is likely to reach.

The linear factor, rather than sub-linear, is attributable to a full-table ordering on weight at retrieval time; adding a secondary index on the weight column would reduce this, and we note it as a straightforward optimization we have not yet applied. We also measured the cost of the supersession filter added in Section 6: introducing the exclusion subqueries on the read path raised mean latency by 12–18% across all three scales (e.g. from 30.1 to 35.2 ms at 10,051 nodes). This is a real, non-zero cost of correctness, and it remains well within the interactive budget.

## 8 Related Work

**LLM memory systems.** MemGPT frames context management as a virtual memory hierarchy, paging information between a limited context window and external storage under LLM control [4]. Mem0 provides a memory architecture that extracts

and consolidates salient facts from conversation, including a graph-structured variant [3]. Asterism shares the goal of persistent conversational memory but differs in emphasis: rather than optimizing recall or context-window management, it treats *prioritization* as the core function and evaluates the weighted graph specifically as an index that supplies salience. Our five-condition comparison is designed to separate that prioritization benefit from the recall benefit those systems primarily target, and further to separate the structure from the instruction to use it.

**Retrieval-augmented generation.** RAG retrieves passages by similarity to a query and conditions generation on them [6]. Similarity retrieval answers "what is relevant to this query;" Asterism's weighting answers a different question, "what is persistently important to this user," accumulated across sessions rather than computed per query. The two are complementary: the index encodes standing priority, not per-query relevance.

**Indexing.** The B-tree [2] is the canonical priority-ordered access structure over a file too large to scan. We borrow its framing directly: the graph is to flat memory what an index is to a heap. The analogy is deliberately used as a design and evaluation lever, not merely as description.

**Biological memory.** The weighting dynamics draw on Hebbian plasticity, in which co-active connections strengthen [1]; the popular "fire together, wire together" phrasing is due to Shatz [7]. We use this as principled motivation for the traversal-strengthening rule, not as a claim of biological fidelity, and describe the mechanics in plain algorithmic terms throughout.

## 9 Conclusion

We presented Asterism, a local-first system that builds a weighted, decaying knowledge graph from conversation history and uses it as a priority index over otherwise flat LLM memory. The framing is the contribution: treating node weight as an index key predicts, correctly, that the structure's benefit over an unstructured list of the same facts is concentrated in decisiveness under ambiguity rather than in recall. A five-condition, three-model evaluation sharpens that claim in three ways: the effect is a property of the structure rather than of any instruction to prioritize, since removing the instruction preserves it and adding it to a flat list does not reproduce it; the added decisiveness is accurate, targeting the genuinely highest-priority item more often than the alternatives rather than committing confidently at random; and the effect is model-dependent, robust across the Claude models we test and absent in the GPT model, which is already decisive without structure. We characterized the two components an automatically-populated index requires — entity resolution, whose embedding-based ceiling we measured rather than hid, and supersession, which demotes contradicted facts from retrieval — and showed the retrieval path scales approximately linearly to ten thousand nodes. We view the negative results, particularly the synonym-resolution ceiling and the GPT non-replication, as among the more useful outputs: they mark exactly where, and for which models, the current design is trustworthy and where it is not.

## Limitations

Our core evaluation uses a single binary metric on twelve priority queries against one seeded graph. Twelve queries is a small sample; we mitigate this by repeating the design across three models and five runs each with reported variance, and the graph−flat gap holds in every Claude run, but a larger, more diverse query set would give a tighter estimate. A single seeded graph is not a population: the graph was authored to resemble a plausible active user, and we have not tested whether the effect generalizes across different persona graphs. The effect is also, on our evidence, Claude-family specific — it is robust in two Claude models and absent in the one GPT model we test — so we do not claim it is a universal property of LLMs; whether other model families pattern with Claude or with GPT is untested and is the most direct extension of this work.

The dependent variable is an automatic classifier. We validate it against human labels on the evaluation data ($\kappa = 0.64$) and show its errors bias the measured gap toward the null, but $\kappa = 0.64$ is substantial rather than near-perfect, so the absolute commit rates carry measurement error concentrated in the treatment condition. Relatedly, our controls isolate structure from instruction but not the two components *within* the structure: the weighted graph carries both a topology (hierarchy, edges) and explicit weight values, and we did not run a condition that holds topology fixed while removing or shuffling the weights. We therefore show that

structure matters, not whether the weights specifically, as opposed to the graph shape, carry the effect; that decomposition is future work and we are careful not to claim it here.

We do not compare end-task accuracy against external memory systems such as Mem0 or MemGPT on a shared benchmark; the decisiveness finding is a characterized, reproducible behavioral effect on this setup rather than a benchmarked performance gain against those systems. The entity-resolution ceiling we report is a limitation of the current unsupervised approach: true synonym pairs are not reliably merged, which can still fragment the index in practice. Supersession is scoped to the same-source/same-relation/different-target case and does not detect subtler contradictions. Our decay is measured in active-session time so a concept does not fade while the user is away; a consequence we note is that decay topology depends on usage frequency, so a daily user prunes old interests quickly while a sparse user's graph decays little in real time and can accumulate stale structure over a year. Retrieval latency, while linear and small in absolute terms, is measured on synthetic graphs and on the retrieval step alone; end-to-end latency is dominated by the model call. Finally, the system is single-user and local by design, and we have not studied multi-user or privacy-preserving extensions.

## Ethics Statement

Asterism is local-first: the graph is stored on the user's own device and no conversational data is transmitted except the user's prompt and the context they choose to inject into their model provider. Deleting the local store removes all derived data. The system infers user priorities from conversation history, and such inferences can be sensitive; because the data and index remain under the user's direct control and can be inspected or deleted, we regard the privacy surface as limited, but any deployment that centralized these graphs would raise concerns that the present design specifically avoids.

## Declaration on Generative AI

The authors used generative AI tools to assist with code for the system implementation and evaluation harness, and to support editing and refinement of the manuscript text (grammar, clarity, and structure). LLMs also appear as objects of study in the evaluation, as the assistant models being measured; those uses are described in the relevant sections. All research design, experimental decisions, analysis, and conclusions are the authors' own, and the authors reviewed and verified all content and take full responsibility for it.

## References


[1] Donald O. Hebb. 1949. *The Organization of Behavior: A Neuropsychological Theory*. Wiley, New York.

[2] Rudolf Bayer and Edward M. McCreight. 1972. Organization and maintenance of large ordered indexes. *Acta Informatica*, 1(3):173–189.

[3] Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. 2025. Mem0: Building production-ready AI agents with scalable long-term memory. *arXiv preprint arXiv:2504.19413*.

[4] Charles Packer, Vivian Fang, Shishir G. Patil, Kevin Lin, Sarah Wooders, and Joseph E. Gonzalez. 2023. MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*.

[5] Shitao Xiao, Zheng Liu, Peitian Zhang, and Niklas Muennighoff. 2023. C-Pack: Packaged resources to advance general Chinese embedding. *arXiv preprint arXiv:2309.07597*.

[6] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474.

[7] Carla J. Shatz. 1992. The developing brain. *Scientific American*, 267(3):60–67.

[8] Aric A. Hagberg, Daniel A. Schult, and Pieter J. Swart. 2008. Exploring network structure, dynamics, and function using NetworkX. In *Proceedings of the 7th Python in Science Conference (SciPy2008)*, pages 11–15, Pasadena, CA.


## A Reproducibility Details

The system is implemented in Python over SQLite, with the graph loaded into NetworkX [8] for analysis. Triple extraction uses a small instruction-tuned model; entity-resolution embeddings use a compact local sentence-embedding model [5] producing 384-dimensional vectors, run offline on CPU. The evaluation graph, the five-condition comparison script and its multi-model runner, the latency benchmark, and the entity-resolution and supersession test suites are included in the public repository.

Latency figures were produced by timing the retrieval function directly with a monotonic clock over 20 runs per scale, on synthetic graphs seeded with weighted domain/theme/concept chains; the real user database is never touched by the benchmark.